Computer Network Orange Problem SDN - Mininet Project

Broadcast Traffic Control using Mininet and POX

Name: Karan Varshney

SRN: PES2UG24CS903

Section: C

Problem Statement

Broadcast traffic is a fundamental part of local area network communication. Protocols such as ARP (Address Resolution Protocol) rely on broadcasts to allow hosts to discover each other's MAC addresses before any unicast communication can take place. Under normal conditions, broadcast traffic is sparse and does not significantly impact network performance. However, when a host continuously floods the network with broadcast packets — either due to a misconfiguration, a network scanning tool, or a malicious activity — every connected device must interrupt its normal processing to inspect and discard those frames. This wastes switch bandwidth, increases CPU load on all hosts, and can degrade overall network throughput.

This project addresses the above problem by implementing a Software Defined Networking (SDN) solution using Mininet and the POX OpenFlow controller. The controller monitors broadcast traffic on a per-source basis and applies a configurable threshold policy: normal broadcasts are forwarded without interference, while hosts that exceed the allowed rate within a given time window are automatically suppressed by installing a high-priority OpenFlow drop rule directly on the switch. This approach ensures that legitimate broadcast traffic is never blocked, and only genuinely excessive sources are controlled.

Objective

- Create a custom Mininet topology with four hosts and one OpenFlow switch.
- Use POX as a remote SDN controller.
- Detect broadcast packets by checking the Ethernet destination MAC address.
- Allow normal ARP broadcast traffic.
- Limit excessive broadcast traffic by installing a high-priority OpenFlow drop rule.
- Validate the behavior using terminal outputs, POX logs, arping, ping, and flow table inspection.

Tools and Technologies Used

Tool	Purpose
Ubuntu on UTM	Linux environment used to run Mininet and POX.
Mininet	Used to emulate hosts, switch, and network links.
Open vSwitch	Acts as the OpenFlow switch in the Mininet topology.
POX	Python-based SDN controller used for packet-in handling and flow rule installation.
OpenFlow 1.0	Protocol used between the controller and switch.
arping	Used to generate ARP broadcast traffic for testing.
ovs-ofctl	Used to verify installed flow rules in the switch.

Theory

Software Defined Networking

Software Defined Networking (SDN) is a network architecture paradigm that separates the control plane from the data plane. In a conventional network, intelligence and packet forwarding decisions are tightly coupled within each individual switch, making it difficult to implement network-wide policies dynamically. SDN overcomes this limitation by centralising the control logic in a dedicated software controller, which communicates with the switches using a standardised protocol such as OpenFlow. The switches become simple forwarding devices that act on the flow rules installed by the controller, while the controller has a global view of the network and can react to events in real time. In this project, the POX framework acts as the SDN controller and Open vSwitch serves as the programmable OpenFlow-capable switch.

Broadcast Traffic

In Ethernet networks, broadcast traffic refers to frames that are addressed to all devices on the local segment simultaneously. This is achieved using the reserved destination MAC address `ff:ff:ff:ff:ff:ff`, which instructs every network interface to receive and process the frame. The most common use of broadcasting is ARP — when a host needs to send data to an IP address but does not yet know the corresponding MAC address, it broadcasts an ARP request asking all devices to identify themselves. While this is essential for normal network operation, excessive broadcast traffic is harmful: every host on the segment must process each frame, and switches must forward broadcasts out of all ports, consuming both CPU and bandwidth resources across the entire network.

Broadcast Control Idea

Since blocking all broadcast traffic would break basic networking protocols like ARP, the controller must take a more selective approach. The core idea is to use a threshold-based rate limiting policy: the controller maintains a per-source timestamp log and counts how many broadcast packets each host has sent within a recent time window. As long as a host's count remains within the configured limit, its broadcasts are treated as legitimate and are flooded normally to all other ports. The moment a host's count exceeds the threshold, the controller classifies it as an excessive broadcaster and responds by installing a high-priority OpenFlow drop rule that matches specifically that source host's broadcasts. This rule is pushed directly to the switch, so subsequent offending packets are dropped in the data plane without ever reaching the controller. After a configurable block timeout, the rule expires and the host is allowed to broadcast again — making the solution both precise and adaptive.

Network Topology

```
h1 ----\

h2 -----\

s1 ----- POX Controller

h3 -----/

h4 ----/
```

Node	IP Address	MAC Address
h1	10.0.0.1	00:00:00:00:00:01
h2	10.0.0.2	00:00:00:00:00:02
h3	10.0.0.3	00:00:00:00:00:03
h4	10.0.0.4	00:00:00:00:00:04

The topology is intentionally kept simple — four hosts connected to a single switch — so that all network traffic flows through one observable point and the broadcast behaviour of each host can be tracked without ambiguity. The switch is configured to use OpenFlow 1.0 and to connect to the POX controller as a remote controller over TCP. When any host sends a packet that the switch does not have a flow entry for, the switch generates a packet-in event and forwards the packet to the controller for processing. The controller then decides whether to flood, forward, or drop the packet and installs the appropriate flow rule.

Controller Logic

The controller is built as a POX component that registers a handler for `PacketIn` events. Each time a packet arrives at the controller, it extracts the source MAC address, the ingress port, and the destination MAC address. For unicast packets (known destination), the controller installs a bidirectional forwarding rule and forwards the current packet immediately. For broadcast packets (destination `ff:ff:ff:ff:ff:ff`), the controller checks a per-source timestamp list, pruning entries older than the time window, and counts the remaining entries to determine the recent broadcast rate of that host.

Parameter	Value
Broadcast limit	5 packets
Time window	10 seconds
Block timeout	20 seconds
Drop rule priority	100

If the broadcast count for a host is within the allowed limit, the controller issues a flood action so the ARP request reaches all other hosts as expected. Once the count exceeds the configured maximum, the controller constructs an `ofp_flow_mod` message and pushes a drop rule to the switch. The rule is matched on three fields: the source MAC address of the offending host, its ingress port, and the broadcast destination MAC — making it precise enough to suppress only that host's excess broadcasts without affecting any other traffic on the network. The rule's hard timeout ensures it is automatically removed from the switch after the block period, restoring normal broadcast ability to the host.

```
msg = of.ofp_flow_mod()

msg.priority = 100

msg.match = of.ofp_match(

in_port=event.port,

dl_src=packet.src,

dl_dst=BROADCAST_MAC

)

event.connection.send(msg)
```

> The drop rule intentionally omits any output action. In OpenFlow, a flow entry that matches a packet but specifies no output port causes the switch to silently discard the packet. No ICMP error or rejection is sent to the source host — the packets are simply absorbed at the switch, keeping the control plane free from further processing overhead.

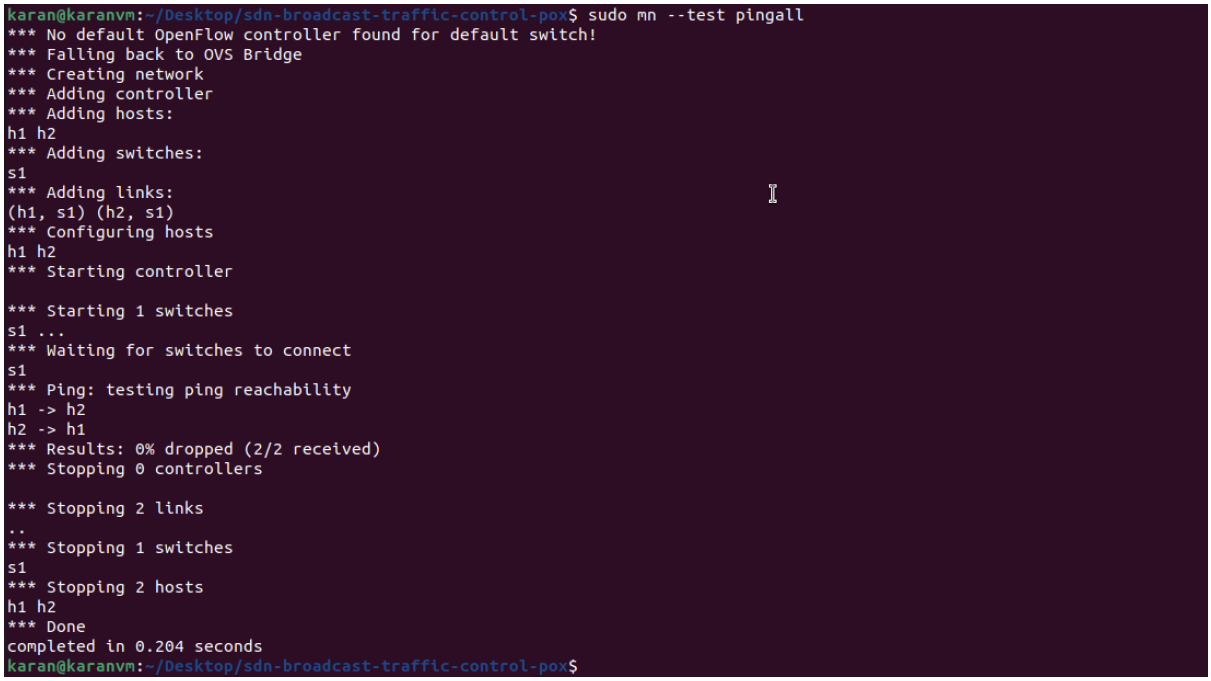
Execution and Screenshots

1. Mininet Verification

```
sudo mn --test pingall
```

Before beginning the main experiment, a baseline verification is performed to confirm that the Mininet environment and Open vSwitch are installed and functioning correctly. The `--test pingall` flag instructs Mininet to automatically create a default two-host topology, run a complete ping reachability test between all host pairs, and then tear down the network. A reported result of 0% packet loss indicates that the emulation environment is healthy and ready for the custom topology.

Screenshot 1: Mininet installation test



```
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$ sudo mn --test pingall
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Stopping 0 controllers

*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 0.204 seconds
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$
```

2. POX Verification

```
cd ~/Desktop/sdn-broadcast-traffic-control-pox/pox

python3 pox.py --help
```

Before launching the controller with the custom module, it is important to verify that POX itself is accessible and correctly set up in the project directory. Running `pox.py --help` prints the POX version string, copyright information, and a summary of supported command-line arguments. Successfully seeing this output confirms that Python 3 can find and execute the POX entry point, which is a necessary prerequisite before loading any custom controller component.

Screenshot 2: POX help output

```
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$ cd ~/Desktop/sdn-broadcast-traffic-control-pox/pox
python3 pox.py --help
POX is a Software Defined Networking controller framework.

The commandline of POX is like:
pox.py [POX options] [C1 [C1 options]] [C2 [C2 options]] ...

Notable POX options include:
  --verbose       Print more debugging information (especially useful for
                  problems on startup)
  --no-openflow   Don't automatically load the OpenFlow module
  --log-config=F  Load a Python log configuration file (if you include the
                  option without specifying F, it defaults to logging.cfg)

C1, C2, etc. are component names (e.g., Python modules).  Options they
support are up to the module.  As an example, you can load a learning
switch app that listens on a non-standard port number by specifying an
option to the of_01 component, and loading the l2_learning component like:
  ./pox.py --verbose openflow.of_01 --port=6634 forwarding.l2_learning

The 'help' component can give help for other components.  Start with:
  ./pox.py help --help
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox/pox$
```

3. Starting the POX Controller

```
python3 pox.py log.level --DEBUG openflow.of_01 --port=6633 broadcast_control
```

The POX controller is started with three components: `log.level --DEBUG` to enable verbose logging so all events are visible in the terminal, `openflow.of_01 --port=6633` to open an OpenFlow 1.0 listener on the standard port, and `broadcast_control` to load the custom broadcast management module. Once started, the controller waits for an OpenFlow switch to connect. All subsequent packet-in events, broadcast detections, flooding decisions, and drop rule installations are printed to the terminal in real time, providing a complete audit trail of the controller's behaviour.

Screenshot 3: POX controller started

```
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox/pox$ cd ~/Desktop/sdn-broadcast-traffic-control-pox/pox
python3 pox.py log.level --DEBUG openflow.of_01 --port=6633 broadcast_control
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:broadcast_control:Broadcast Traffic Control Controller started
INFO:broadcast_control:Policy: max 5 broadcast packets per 10 seconds
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.10.12/Mar 3 2026 11:56:32)
DEBUG:core:Platform is Linux-5.15.0-174-generic-aarch64-with-glibc2.35
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
```

4. Starting Custom Mininet Topology

```
sudo mn --custom topos/broadcast_topo.py --topo broadcasttopo \

--controller remote,ip=127.0.0.1,port=6633 \

--switch ovsk,protocols=OpenFlow10

nodes

net
```

The custom Mininet topology script `broadcast_topo.py` creates four hosts (h1–h4) connected to a single Open vSwitch (s1), which is in turn configured to communicate with the POX controller at 127.0.0.1:6633 using OpenFlow 1.0. Once the topology is running, the Mininet CLI becomes available. The `nodes` command confirms that all six network entities — c0, h1, h2, h3, h4, and s1 — are active, and the `net` command verifies the link structure, showing that each host is connected to s1 through a dedicated Ethernet interface.

Screenshot 4: Custom topology and links

```
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$ cd ~/Desktop/sdn-broadcast-traffic-control-pox
sudo mn --custom topos/broadcast_topo.py --topo broadcasttopo --controller remote,ip=127.0.0.1,port=6633 --switch ovsk,protocols=OpenFlow10
[sudo] password for karan:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
c0 h1 h2 h3 h4 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0
c0
mininet>
```

5. Normal Connectivity Test

```
h1 ping -c 2 h2
```

Before testing broadcast-specific behaviour, it is essential to establish that basic end-to-end connectivity is working correctly under the SDN controller. This test sends two ICMP echo requests from h1 to h2. The first packet triggers a packet-in event, causing the controller to learn the MAC-to-port mapping for both hosts and install bidirectional forwarding rules. The second packet is handled directly by the switch using the installed rules. A 0% packet loss result confirms that the controller is correctly managing unicast traffic and is not inadvertently blocking legitimate communication.

Screenshot 5: Normal ping from h1 to h2

```
mininet> h1 ping -c 2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.53 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.551 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.551/1.541/2.531/0.990 ms
mininet>
```

6. Normal Broadcast Test

```
h1 arping -c 3 10.0.0.2
```

The `arping` tool operates at Layer 2 and sends ARP request frames directly using the Ethernet broadcast destination address. By sending only 3 packets, this test keeps the broadcast count for h1 well within the configured limit of 5 packets per 10-second window. Each packet-in event is processed by the controller, which increments h1's broadcast counter, determines that the threshold has not been crossed, and floods the frame out of all other switch ports. The arping output showing 3 packets transmitted and 3 received confirms that normal broadcast traffic is forwarded without any interference.

Screenshot 6: Normal ARP broadcast is successful

```
mininet> h1 arping -c 3 10.0.0.2
ARPING 10.0.0.2
42 bytes from 00:00:00:00:00:02 (10.0.0.2): index=0 time=3.206 msec
42 bytes from 00:00:00:00:00:02 (10.0.0.2): index=1 time=1.399 msec
42 bytes from 00:00:00:00:00:02 (10.0.0.2): index=2 time=1.202 msec

--- 10.0.0.2 statistics ---
3 packets transmitted, 3 packets received,   0% unanswered (0 extra)
rtt min/avg/max/std-dev = 1.202/1.936/3.206/0.902 ms
mininet>
```

POX Logs for Normal Broadcast

The POX controller terminal logs a detailed entry for every broadcast packet-in event it handles. Each log line identifies the source MAC address, the current broadcast count for that source, and the action taken. For the 3-packet test, all entries are marked as BROADCAST detected and allowed, with count values of 1/5, 2/5, and 3/5 respectively. This confirms that the packet-in handler is correctly classifying broadcast traffic, maintaining accurate per-source counters, and making the right forwarding decisions — all while staying below the threshold.

Screenshot 7: POX logs for allowed broadcast packets

```
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:02 -> 00:00:00:00:00:01 out_port=1
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:01 -> 00:00:00:00:00:02 out_port=2
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:02 -> 00:00:00:00:00:01 out_port=1
INFO:broadcast_control:UNKNOWN destination flooded: dst=33:33:00:00:00:02
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:02 -> 00:00:00:00:00:01 out_port=1
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:01 -> 00:00:00:00:00:02 out_port=2
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=1/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:UNICAST forwarded: 00:00:00:00:00:02 -> 00:00:00:00:00:01 out_port=1
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=2/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=3/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:UNKNOWN destination flooded: dst=33:33:00:00:00:02
INFO:broadcast_control:UNKNOWN destination flooded: dst=33:33:00:00:00:02
INFO:broadcast_control:UNKNOWN destination flooded: dst=33:33:00:00:00:02
```

7. Excessive Broadcast Test

```
h1 arping -c 10 10.0.0.99
```

To trigger the broadcast control mechanism, h1 is made to send 10 consecutive ARP requests targeting IP address 10.0.0.99 — an address that does not exist anywhere in the topology. Since no host can respond, h1 receives no ARP replies and continues broadcasting for each of the 10 attempts. All 10 packets time out on the arping side, and each one reaches the POX controller as a packet-in event. This allows us to observe exactly how the controller's broadcast counter escalates and at what point the threshold of 5 packets within the 10-second window is crossed.

Screenshot 8: Repeated ARP requests to a non-existing host

```
mininet> h1 arping -c 10 10.0.0.99
ARPING 10.0.0.99
Timeout
Timeout
Timeout
Timeout
Timeout
Timeout
Timeout
Timeout
Timeout
Timeout

--- 10.0.0.99 statistics ---
10 packets transmitted, 0 packets received, 100% unanswered (0 extra)

mininet>
```

Controller Detects Limit Exceeded

```
BROADCAST detected: count=6/5

LIMIT EXCEEDED: installed DROP rule
```

The POX log clearly shows the escalation of h1's broadcast count across consecutive packets. The first five broadcasts are logged as BROADCAST detected and allowed. On the sixth packet, the count reaches 6/5, exceeding the limit of 5, and the controller logs a LIMIT EXCEEDED warning. At this point the controller does not flood the packet. Instead, it

constructs and sends an `ofp_flow_mod` message to the switch with priority 100 and no output action. All subsequent broadcast frames from h1 are then matched and dropped directly at the switch, without generating any further packet-in events — significantly reducing the load on the controller.

Screenshot 9: Controller detects limit exceeded and installs drop rule

```
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=1/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=2/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=3/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=4/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=5/5
INFO:broadcast_control:BROADCAST allowed and flooded: src=00:00:00:00:00:01
INFO:broadcast_control:BROADCAST detected: src=00:00:00:00:00:01 in_port=1 count=6/5
WARNING:broadcast_control:LIMIT EXCEEDED: installed DROP rule for broadcast src=00:00:00:00:00:01 port=1 timeout=20s
```

8. Flow Table Verification

```
sh sudo ovs-ofctl -O OpenFlow10 dump-flows s1
```

To independently verify that the drop rule was actually installed in the switch's data plane — and not merely logged by the controller — the flow table is inspected directly using `ovs-ofctl`. This command queries the Open vSwitch and prints all active flow entries. The output includes an entry with priority=100 that matches h1's source MAC address, its ingress port, and the broadcast destination MAC `ff:ff:ff:ff:ff:ff`, with `actions=drop`. This provides definitive proof that the OpenFlow control mechanism is functioning end-to-end, from the controller's policy decision all the way to the switch's forwarding hardware.

Screenshot 10: OpenFlow drop rule in switch flow table

```
mininet> sh sudo ovs-ofctl -O OpenFlow10 dump-flows s1
 cookie=0x0, duration=16.445s, table=0, n_packets=15, n_bytes=870, idle_timeout=20, hard_timeout=20, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:
01,dl_dst=ff:ff:ff:ff:ff:ff actions=drop
mininet>
```

9. Cleanup

```
exit

sudo mn -c
```

After completing all tests, the Mininet session is terminated cleanly. The `exit` command shuts down the Mininet CLI and stops all emulated hosts and links. Running `sudo mn -c` performs a deeper cleanup pass, removing any residual Open vSwitch bridges, flushing kernel network namespaces, and killing any lingering controller or datapath processes that may still be running in the background. This step is important because leftover virtual interfaces or OVS state from a previous run can cause errors or unexpected behaviour in subsequent experiments.

Screenshot 11: Mininet cleanup

```
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 4 links
....
*** Stopping 1 switches
s1
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
completed in 658.577 seconds
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/
null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /d
ev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
karan@karanvm:~/Desktop/sdn-broadcast-traffic-control-pox$
```

Results Summary

Test	Expected Result	Observed Result
Mininet verification	Mininet runs and pingall succeeds	Successful
POX verification	POX help output appears	Successful
Topology creation	Four hosts connect to one switch	Successful
Normal ping	h1 can reach h2	Successful
Normal broadcast	ARP broadcast is allowed	Successful
Excessive broadcast	Controller detects threshold violation	Successful
Flow rule check	Drop rule appears in flow table	Successful

Conclusion

This project successfully demonstrates how Software Defined Networking can be applied to solve a practical network management problem. By leveraging the OpenFlow protocol and the POX controller, a dynamic broadcast traffic control system was designed and validated entirely within an emulated environment. The system correctly distinguishes between normal and excessive broadcast behaviour on a per-host basis, allowing legitimate ARP traffic to pass freely while automatically suppressing sources that exceed the configured threshold. The control mechanism operates without any manual intervention and without disrupting other hosts on the network.

All nine experimental steps produced the expected results, which are summarised in the Results Summary table above. The most significant outcome is the verification of the OpenFlow drop rule directly in the switch's flow table, which confirms that the policy decision made by the controller is faithfully enforced in the data plane. This end-to-end validation — from packet-in event handling in the controller to flow rule installation in the switch — demonstrates a complete and working SDN-based broadcast control solution that satisfies all the requirements of the Orange Problem assignment.

References

- Mininet Overview: https://mininet.org/overview/
- Mininet Walkthrough: https://mininet.org/walkthrough/
- POX Controller Repository: https://github.com/noxrepo/pox
- OpenFlow match-action rule concept